Unidad II · Actividad 4

Configuración de permisos para diferentes roles

Fundamentos de Arquitectura de Software y Desarrollo Backend

12 al 16 de octubre · Actividad #8 · Gestión de permisos por roles

Esta actividad sigue la actividad 3. Allá quedaron los roles, los permisos y la lista que une a los dos. Aquí se usa ese proyecto para configurar qué puede hacer cada puesto y para comprobar que el sistema obedece esa configuración.

El trabajo sigue en el Sistema General de Ingresos. Sirve el mismo Python, la misma base MySQL, el mismo GraphQL y, si el equipo lo dejó armado, el mismo Docker Compose.

Al terminar vas a poder leer la lista de un rol, cambiarla una sola vez y ver el efecto en dos personas de ese puesto. También vas a cambiar el puesto de una persona sin tocar la lista de los demás.

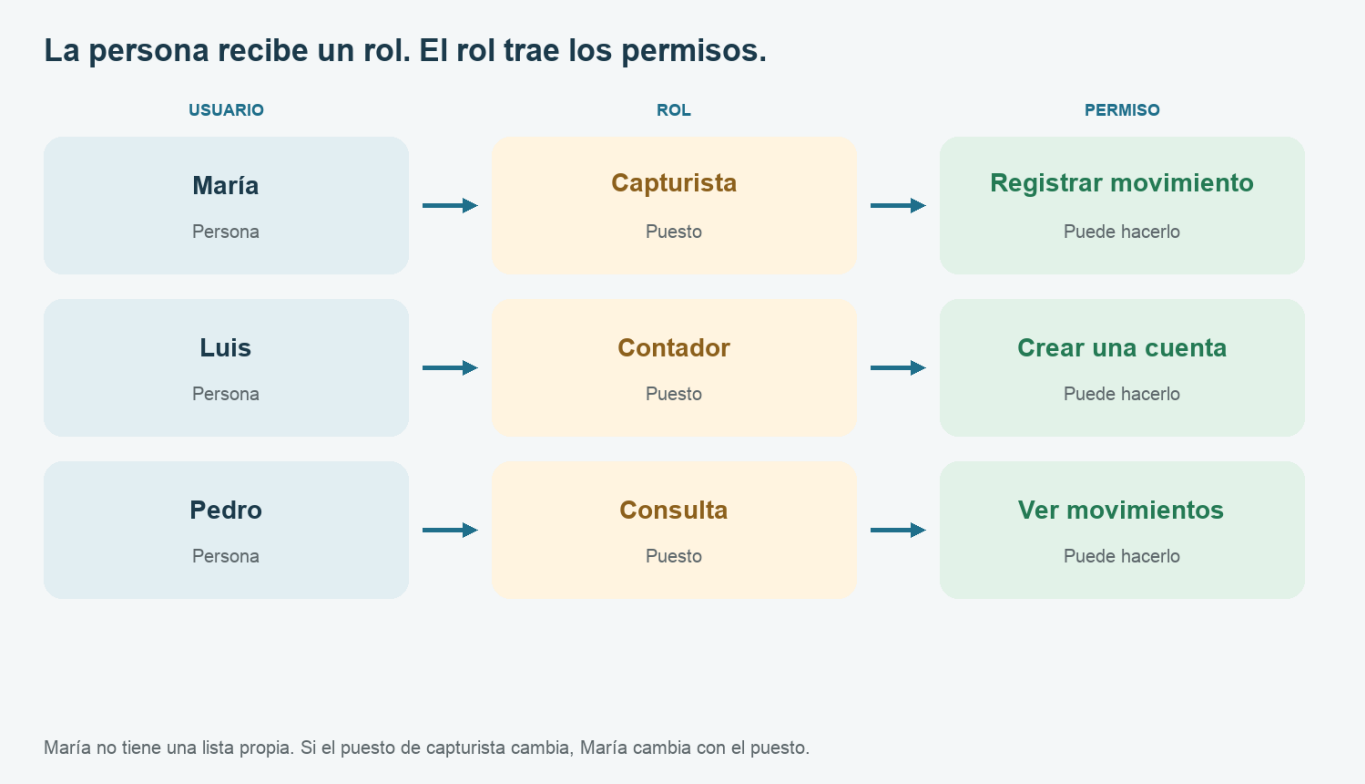
Usuarios, roles y permisos

Hay tres piezas. Conviene tenerlas separadas desde el principio.

Pieza	Pregunta que contesta	Ejemplo
Usuario	¿Quién es?	María
Rol	¿Qué puesto tiene en esta empresa?	Capturista
Permiso	¿Qué acción está permitida?	Registrar un movimiento

El usuario es la persona. El rol es el puesto. El permiso es una acción concreta: crear una cuenta, registrar un movimiento, invitar a un usuario, ver los movimientos.

La persona recibe un rol. El rol trae los permisos. María no guarda una lista propia de acciones.



En el sistema de la actividad 3 esos nombres ya tienen lugar:

- El usuario vive en `User`.
- Su lugar en una empresa vive en `CompanyUser`.
- El puesto de ese lugar vive en `company_user_roles`.
- La lista de acciones de cada puesto vive en `role_permissions`.

Cómo se gestiona

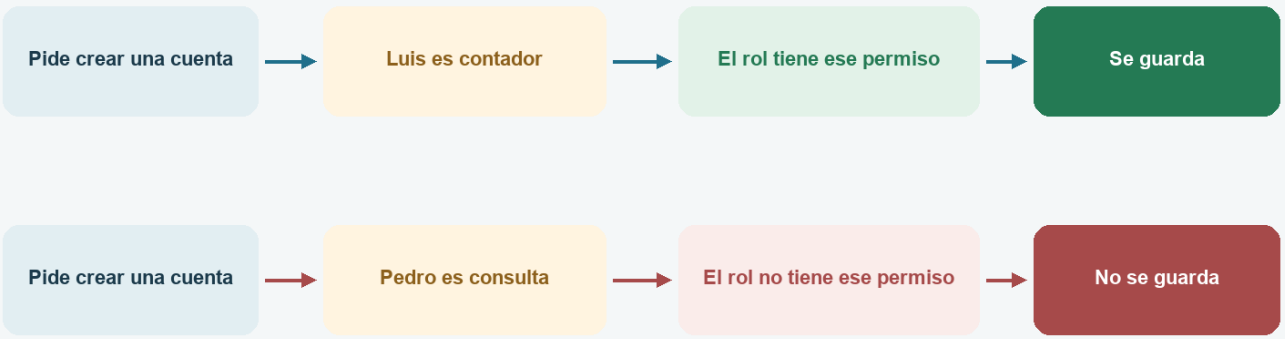
Configurar permisos es decidir la lista de cada puesto y después decir qué puesto tiene cada persona. El orden importa.

1. Los puestos y las acciones ya existen en un catálogo. En este sistema son cuatro roles y diez permisos. No se inventa un permiso nuevo por cada persona.
2. Cada rol tiene su lista. El capturista tiene unas acciones. El contador tiene otras. La misma acción puede estar en varios roles.
3. Cada persona, dentro de una empresa, tiene un solo rol activo. Si pasa de capturista a contador, el puesto anterior se desactiva.
4. Cuando alguien pide una acción, el servidor mira el rol de esa persona y busca la acción en la lista de ese rol. Si está, guarda. Si falta, rechaza y no guarda.

Así se administra el acceso:

- Un cambio de puesto se hace en el rol. Si mañana todos los capturistas pueden crear cuentas, se agrega esa acción al rol Capturista. María y José quedan incluidos, sin editar a cada uno.
- Una persona cambia de puesto con `assignRole`. Su lista nueva es la del puesto nuevo. La lista del puesto anterior sigue igual para las demás personas.
- Ver y escribir son dos permisos. Poder ver las cuentas no incluye poder crearlas.
- La lista no se borra. Si se quita una acción, la pareja queda inactiva. El historial de la configuración sigue en la tabla.

El sistema pregunta el rol antes de guardar



Luis sí puede crear la cuenta.

Pedro pide la misma acción y el sistema la rechaza. La cuenta no se crea.

La acción es la misma. Cambia el rol de quien la pide.

Luis y Pedro piden la misma acción: crear una cuenta. Luis es contador y su rol tiene ese permiso, así que la cuenta se guarda. Pedro es consulta y su rol no tiene ese permiso, así que la cuenta no se crea.

La pregunta vive en Python, en `exigir_permiso`, antes de guardar. GraphQL entrega la petición. El permiso lo decide el servidor.

Cinco personas, una empresa

Los ejemplos usan una sola empresa y cinco personas. Ana, Luis, María y Pedro cubren los cuatro puestos. José entra después, como segundo capturista, para comprobar que un cambio de rol alcanza para dos personas.

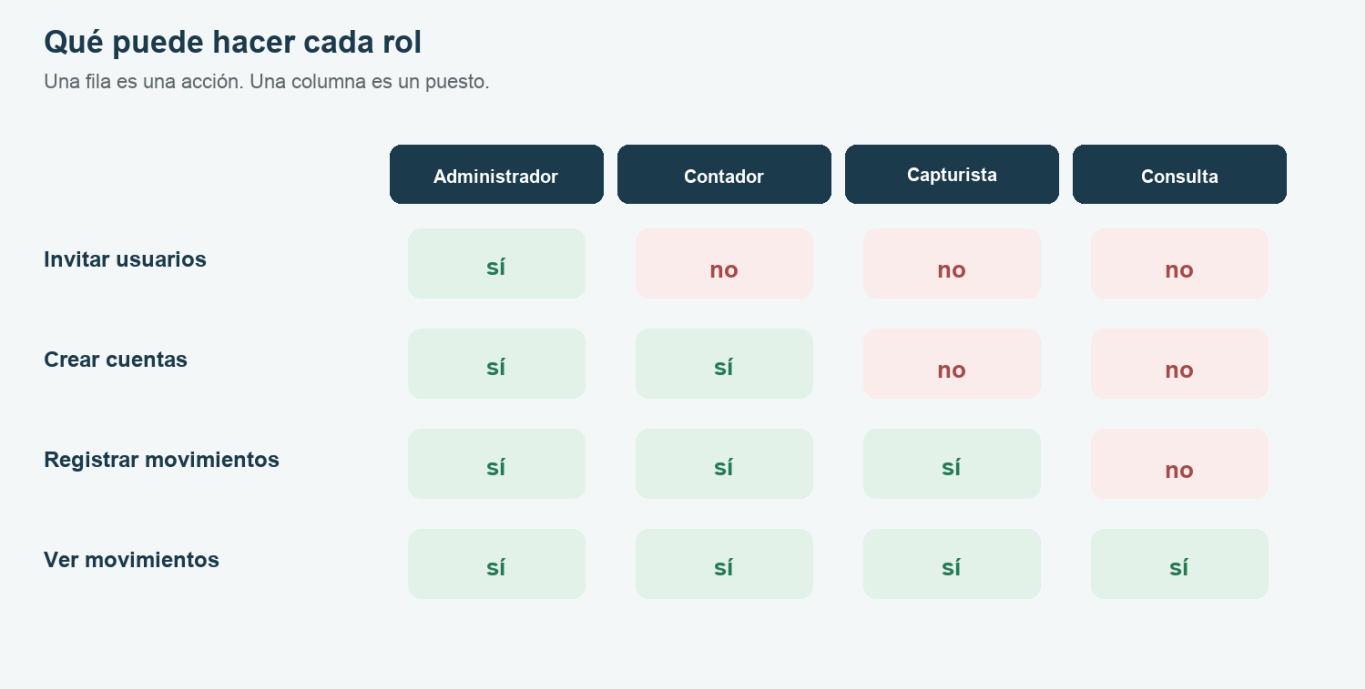


Persona	Rol al empezar	Para qué se usa en la prueba
Ana	Administrador	Cambia la lista de los roles
Luis	Contador	Su lista no debe moverse cuando otra persona cambia de puesto
María	Capturista	Recibe el cambio que se hace en el rol
José	Capturista	Recibe el mismo cambio, sin editar su ficha
Pedro	Consulta	Cambia de puesto. La lista de consulta se queda como estaba

Ana sale de `createCompanyAdmin`. Esa membresía sigue con `isAdmin` en true y con el rol `ADMINISTRADOR`. Las otras cuatro salen de `createCompanyUser` y de `assignRole`.

Qué puede hacer cada rol

La figura de abajo resume cuatro acciones del día a día. La tabla que sigue es la lista completa. Es la misma configuración de la actividad 3. Esta actividad empieza con esa lista ya cargada.



Permiso	Qué permite	Administrador	Contador	Capturista	Consulta
`company.users.read`	Ver los usuarios de la empresa	sí	sí	no	sí
`company.users.write`	Invitar usuarios y desactivar su lugar	sí	no	no	no
`accounts.read`	Ver cuentas	sí	sí	sí	sí
`accounts.write`	Crear cuentas	sí	sí	no	no
`concepts.read`	Ver conceptos	sí	sí	sí	sí
`concepts.write`	Crear conceptos y asignarlos a una cuenta	sí	sí	no	no
`transactions.read`	Ver movimientos	sí	sí	sí	sí
`transactions.write`	Registrar un ingreso o un egreso	sí	sí	sí	no
`roles.read`	Ver roles y la lista de cada uno	sí	sí	no	no
`roles.write`	Cambiar la lista de un rol y asignar un rol	sí	no	no	no

Leída por puesto:

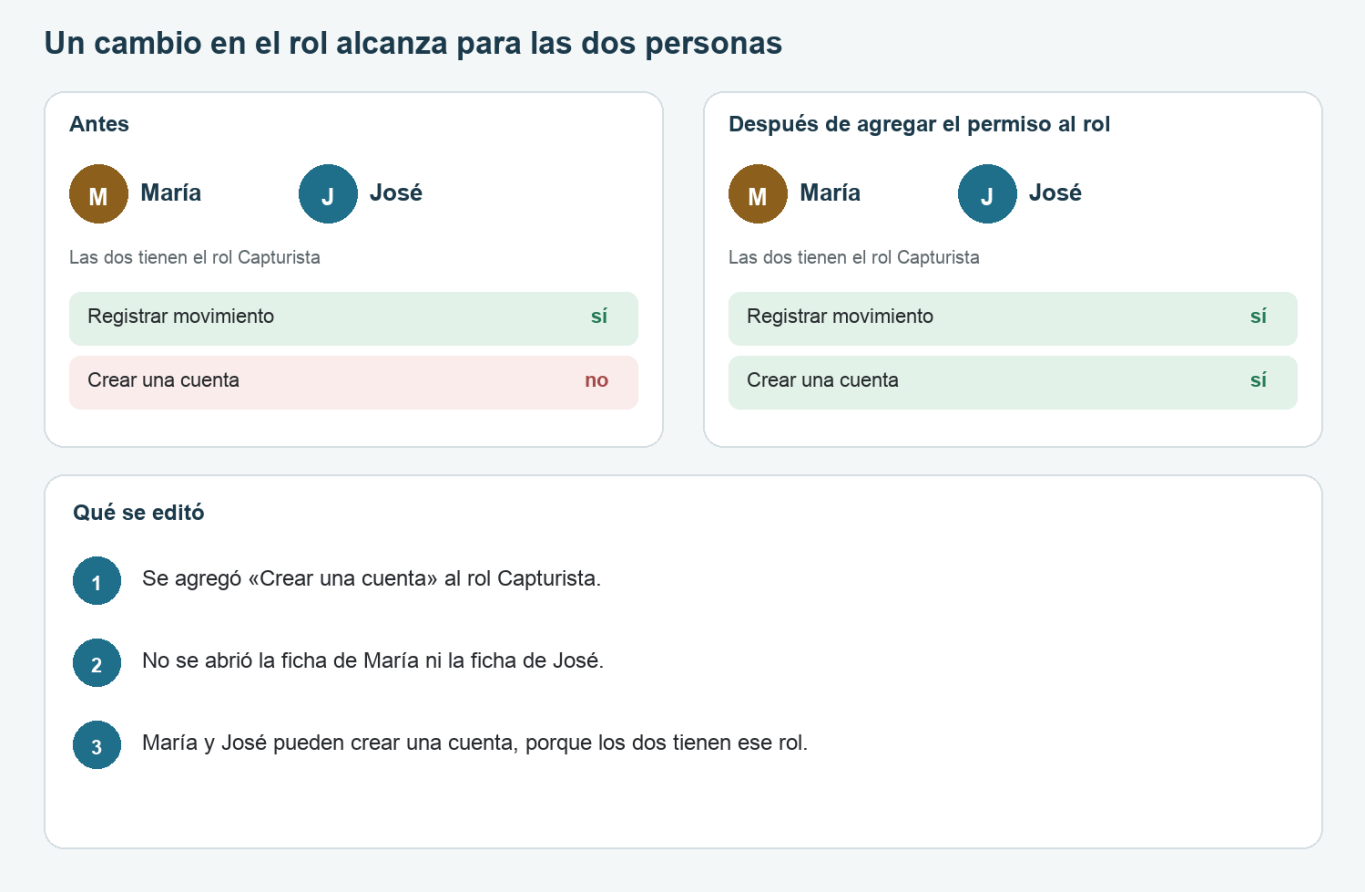
- Ana configura el acceso e invita usuarios.
- Luis arma cuentas y conceptos, y también registra movimientos. No invita usuarios ni cambia la lista de permisos.
- María y José registran movimientos. Pueden ver cuentas y conceptos para elegirlos. No los crean.
- Pedro ve la operación. No escribe.

`accounts.read` no incluye `accounts.write`. Quien crea cuentas también necesita verlas, y por eso el contador tiene las dos.

Un cambio se hace en el rol

La empresa decide que los capturistas también puedan crear cuentas. El cambio es uno: agregar `accounts.write` al rol Capturista.

María y José no se editan. Los dos pueden crear cuentas porque los dos tienen ese rol. Si más tarde se quita el permiso del rol, los dos dejan de poder.



Eso es distinto de cambiarle el puesto a una sola persona. Si Pedro pasa de consulta a capturista, se usa `assignRole`. Pedro recibe la lista del capturista. Luis sigue como contador y su lista no se toca. La lista de consulta tampoco cambia: simplemente Pedro ya no la usa.

Si solo María debiera crear cuentas, y José no, ese caso no se resuelve agregando el permiso al rol Capturista. Los dos quedarían con la misma lista. Haría falta otro puesto. En esta actividad no se crea ese puesto: se practica el cambio que sí corresponde al rol.

Dónde se hace el cambio

Se sigue trabajando en la carpeta del Sistema General de Ingresos. No se abre un proyecto nuevo.

```
ingresos/
  database.py
  models.py
  schema.py
```

Si los archivos tienen otros nombres, se conservan. El `docker-compose` de las actividades anteriores sigue levantando MySQL. No hace falta un contenedor nuevo.

La actividad 3 ya dejó estas operaciones. Aquí se usan, no se vuelven a diseñar.

Qué se quiere hacer	GraphQL
Ver los roles	`roles`
Ver los permisos	`permissions`
Ver la lista de un rol	`rolePermissions(roleId:)`

Qué se quiere hacer	GraphQL
Agregar un permiso a un rol	`assignPermissionToRole`
Quitar un permiso de un rol	`removePermissionFromRole`
Darle un rol a una persona	`assignRole`

`assignPermissionToRole` rechaza la pareja si ya está activa. `removePermissionFromRole` desactiva la pareja. No borra la fila. `assignRole` deja un solo rol activo en esa persona.

En GraphQL, agregar «crear cuenta» al capturista se ve así. Los id salen de `roles` y de `permissions`.

```
mutation {
  assignPermissionToRole(
    input: { roleId: "ID_CAPTURISTA", permissionId: "ID_ACCOUNTS_WRITE" }
    actorCompanyUserId: "ID_DE_ANA"
  ) {
    role {
      code
    }
    permission {
      code
    }
    isActive
  }
}
```

`actorCompanyUserId` es el mismo dato de la actividad 3: la membresía con la que se prueba en GraphiQL. Si en tu proyecto ese dato va dentro del `input`, déjalo donde ya está. El servidor no cree el permiso que el cliente escriba en el texto. Busca el rol de esa membresía y lee la lista.

El trabajo real ocurre en Python: `exigir_permiso` corre antes de guardar. Si el rol no tiene el permiso, no hay `commit` y la tabla no gana la fila.

Para mirar la lista en MySQL:

```
SELECT r.code AS rol, p.code AS permiso, rp.is_active
FROM role_permissions rp
JOIN roles r ON r.id = rp.role_id
JOIN permissions p ON p.id = rp.permission_id
ORDER BY r.code, p.code;
```

Actividad para entregar

Configuración de permisos para diferentes roles del sistema.

Continúa el Sistema General de Ingresos de la actividad 3. Si los roles, los permisos o `exigir_permiso` todavía no están, termínalos primero con ese documento. Esta actividad no reemplaza esas tablas: las configura y demuestra el efecto.

Usa las herramientas que el proyecto ya tiene: Python, Strawberry, GraphQL, MySQL y Docker Compose.

Prepara una empresa con Ana, Luis, María, José y Pedro, en los roles de la tabla de arriba. Pueden ser usuarios nuevos o usuarios que el equipo ya tenía, con el rol que indica esta actividad.

Parte 1. La configuración inicial

Consulta la lista de los cuatro roles y compárala con la tabla de este documento.

- `roles` muestra `ADMINISTRADOR`, `CONTADOR`, `CAPTURISTA` y `CONSULTA`.

- `permissions` muestra los diez códigos.
- `rolePermissions` del capturista incluye `transactions.write` y no incluye `accounts.write`.
- `rolePermissions` de consulta no incluye ningún permiso que termine en `.write`.
- `rolePermissions` del contador incluye `accounts.write` y no incluye `roles.write`.

Si un rol está vacío, carga la lista como indica la actividad 3. Un reinicio no debe duplicar roles ni permisos.

Parte 2. Un permiso nuevo en el rol, dos personas

María y José son capturistas.

1. Con María, `createAccount` se rechaza.
2. Con José, `createAccount` se rechaza.
3. Con Ana, `assignPermissionToRole` agrega `accounts.write` al rol `CAPTURISTA`.
4. Con María, `createAccount` guarda la cuenta.
5. Con José, `createAccount` guarda la cuenta. José no se editó.
6. Con Ana, `removePermissionFromRole` quita `accounts.write` del capturista.
7. María y José vuelven a recibir rechazo. La fila de la pareja queda inactiva. No se borra.

`createAccount` es la misma mutación de las actividades anteriores. El permiso se decide con `exigir_permiso` y el `actorCompanyUserId` de quien pide la cuenta.

Parte 3. Cambiar el puesto de una persona

Pedro empieza como consulta.

1. Con Pedro, `createTransaction` se rechaza y la tabla de movimientos no gana esa fila.
2. Con Ana, `assignRole` deja a Pedro como `CAPTURISTA`. El rol anterior queda inactivo.
3. Con Pedro, `createTransaction` guarda el movimiento.
4. `rolePermissions` del contador sigue igual: Luis todavía puede crear una cuenta.
5. `rolePermissions` de consulta sigue sin permisos `.write`. Pedro ya no usa ese rol, pero la lista del rol no cambió.
6. Pedro tiene un solo rol activo.

Parte 4. Quién puede cambiar la lista

Cambiar la lista pide `roles.write`. Ese permiso lo tiene el administrador.

1. Con José, `assignPermissionToRole` intenta agregar `concepts.write` al rol `CONSULTA`. La operación se rechaza.
2. En MySQL, la lista activa no cambió.
3. Con Ana, la misma operación sí agrega el permiso.
4. Con Ana, `removePermissionFromRole` lo quita otra vez. Consulta vuelve a quedar sin permisos de escritura.

Al cerrar la actividad, la lista de los cuatro roles coincide de nuevo con la tabla de este documento.

Comprobación

Arranca el servidor como ya lo vienen haciendo y abre `/graphql`. Una captura por punto, y junto a cada captura una frase: qué regla demuestra.

- La lista del capturista, antes del cambio, sin `accounts.write`.
- `createAccount` rechazado para María y para José.
- `assignPermissionToRole` hecho por Ana.
- `createAccount` aceptado para María y para José, después del cambio.
- La pareja desactivada en MySQL, no borrada, y el rechazo de los dos al quitar el permiso.
- Pedro rechazado como consulta y aceptado después de `assignRole`.

- La lista del contador, igual antes y después del cambio de Pedro.
- El intento de José de cambiar la lista, rechazado, y el de Ana, aceptado.

Reinicia el servidor al final y confirma que los cuatro roles y los diez permisos no se duplicaron.

Entrega

- El código del Sistema General de Ingresos, el mismo de la actividad 3, con la configuración probada.
- Capturas de los puntos de la comprobación.
- Junto a cada captura, una frase con la regla que demuestra.
- El `SELECT` de `role_permissions` al final, coincidiendo con la tabla de este documento.

Criterios

- El trabajo continúa el proyecto de la actividad 3 de la Unidad 2. No es un proyecto nuevo.
- La lista vive en `role_permissions`. No es una columna nueva de `User`.
- Agregar `accounts.write` al capturista cambia a María y a José.
- Quitar ese permiso los vuelve a dejar fuera, y la fila queda inactiva.
- `assignRole` deja un solo rol activo.
- Cambiar el puesto de Pedro no modifica la lista del contador ni la lista de consulta.
- José no puede cambiar la lista. Ana sí.
- Un rechazo no guarda la fila: no hay `commit` si el permiso falta.
- Los datos de una empresa no se mezclan con los de otra.
- Al entregar, la lista de los cuatro roles vuelve a coincidir con la tabla de este documento.